Experiment-2

What is Shell?

In an operating system, a shell acts as an intermediary between the user and the operating system kernel. It provides a user interface for interacting with the system, allowing users to execute commands, manage files, and run programs. Essentially, it is a command-line interpreter that takes user input, translates it into instructions the kernel can understand, and then executes those instructions.

What is Shell scripting?

Shell scripting is writing a series of commands for a Unix/Linux shell to execute. A shell script is a text file containing these commands, and it is used to **automate repetitive tasks**, run programs, manipulate files, and manage system operations.

1) Aim

Shell Script to check whether the given number is even or odd.

Algorithm

1. Start
2. Read num
3. If num % 2 == 0 then
   a. Print “num is even number”

   Else
   a. Print “num is odd number”
4. Stop

Program

```
#!/bin/bash
    read -p "Enter the number:" num
    if [ ((num%2)) − eq0];then
    echo "numisevennumber”
    else
    echo "numisoddnumber”
    fi
```

Output

```
student@cselab2-10:~/1234$ gedit evenorodd.sh
student@cselab2-10:~/1234$ ./evenorodd.sh
Enter the number:10
10 is even number
student@cselab2-10:~/1234$ ./evenorodd.sh
Enter the number:111
111 is odd number
```

2)Aim: Shell script to find whether the given number is prime or not.

Algorithm:

1. Start
2. Read num
3. If num $\leq$ 1 then
    a. Print "Neither Prime nor Composite"
    b. Stop
4. For i = 2 to num - 1 do
    a. If num % i == 0 then
        i. Print "num is not prime number"
        ii. Stop
5. End For
6. Print "num is prime number"
7. Stop

Program:

```
#!/bin/bash
read -p "Enter the number:" num
if [ $num -le 1 ] ;
then
echo "Neither Prime nor composite"

else
```

```
for ((i=2;i<num;i++))
do

if [ $((num%i)) -eq 0 ] ;
then

echo "$num is not prime number "

exit 0

fi

done

echo "$num is prime number"

fi
```

Output:

```
student@cselab2-10: /1234$ gedit prime.sh
student@cselab2-10: /1234$ chmod 777 prime.sh
student@cselab2-10: /1234$ ./prime.sh
Enter the number:10
10 is not prime number
student@cselab2-10: /1234$ ./prime.sh
Enter the number:313
313 is prime number
student@cselab2-10: /1234$ ./prime.sh
Enter the number:-89
Neither Prime nor composite
```

3)Aim: Shell script to find factorial of a number.

Algorithm:

1. Start
2. Read num
3. Set fact ← 1
4. If num > 0 then
    a. For i ← 1 to num do
        fact ← fact * i
    End For
    b. Print "Factorial of num is fact"

    Else

    a. Print "Factorial doesn't exist for negative numbers"

5. Stop

Program:

```
#!/bin/bash

read -p "Enter your number:" num

fact=1

if [ $num -gt 0 ];
then

for ((i=1;i<=num;i++))
do

fact=$((fact*i))

done

echo "Factorial of $num is $fact"

else

echo "Factorial doesn't exists for negative numbers"

fi
```

Output:

```
student@cselab2-10: /160123733150gedit factorial.sh
student@cselab2-10: /160123733150chmod777 factorial.sh
student@cselab2-10: /160123733150./factorial.sh
Enter your number:5
Factorial of 5 is 120
student@cselab2-10: /160123733150gedit factorial.sh
student@cselab2-10: /160123733150./factorial.sh
Enter your number:12
Factorial of 12 is 479001600
student@cselab2-10: /160123733150./factorial.sh
Enter your number:-5
Factorial doesn't exists for negative numbers
```

4)Aim: Shell script to check whether the given number is Armstrong or not.

Algorithm:

1. Start
2. Read num
3. Set count ← 0, res ← 0, temp ← num
4. Step 1: Count the number of digits
5. While temp > 0 do
    a. count ← count + 1
    b. temp ← temp / 10
6. End While
7. Set temp ← num
8. Step 2: Calculate sum of digits raised to power 'count'
9. While temp > 0 do
    a. rem ← temp % 10
    b. power ← 1

c. For i ← 1 to count do
power ← power * rem
End For

d. res ← res + power

e. temp ← temp / 10

10. End While

11. If res == num then

a. Print "num is an Armstrong number"

Else

a. Print "num is not an Armstrong number"

12. Stop

Program:

```
#!/bin/bash

read -p "Enter the number:" num

count=0
temp=$num
res=0

while [ $temp -gt 0 ]
do
count=$((count+1))
temp=$((temp/10))
done

temp=$num

while [ $temp -gt 0 ]
do
rem=$((temp%10))
res=$((res+rem**count))
temp=$((temp/10))
done

if [ $res -eq $num ] ;
then
echo "$num is an armstrong number"
else
echo "$num is not an armstrong number"
fi
```

Output:

```
Enter the number:153
153 is an armstrong number
```

```
Enter the number:1634
1634 is an armstrong number
```

```
Enter the number:79
79 is not an armstrong number
```

5)Aim: Shell Script to check whether a number is palindrome or not.

Algorithm:

1. Start
2. Read num
3. Set temp ← num
4. Set n ← 0
5. While temp > 0 do
   a. rem ← temp % 10
   b. n ← n * 10 + rem
   c. temp ← temp / 10
6. End While
7. If num == n then
   a. Print "num is a Palindrome number"

   Else
   a. Print "num is not a Palindrome number"
8. Stop

Program:

```
#!/bin/bash

read -p "Enter the number:" num

temp=$num

n=0

while [ $temp -gt 0 ]
do

rem=$((temp%10))

n=$((n*10+rem))

temp=$((temp/10))

done

if [ $num -eq $n ] ;
then

echo "$num is a Palindrome number."

else

echo "$num is not a palindrome number."

fi
```

Output:

```
Enter the number:143
143 is not a palindrome number.
```

```
Enter the number:1441
1441 is a palindrome number
```

6)Aim: Shell script program to print Fibonacci series upto 'n' terms.
Algorithm:

1. Start
2. Read num (number of terms to generate)
3. Initialize:

$$a \leftarrow 0$$

$b \leftarrow 1$

4. Print a and b
5. For i from 2 to num - 1 do
    a. temp ← a + b
    b. a ← b
    c. b ← temp
    d. Print b
6. End For
7. Stop

Program:

```
#!/bin/bash

read -p "Enter the number of terms: " num

a=0
b=1

echo -n "$a $b "

for ((i=2; i<num; i++))
do
    temp=$((a + b))
    a=$b
    b=$temp
    echo -n "$b "
done
```

Output:

```
Enter the number. of terms:6
0 1 1 2 3 5
```

```
Enter the number. of terms:10
0 1 1 2 3 5 8 13 21 34
```